

Context-aware prompting for collaborative problem solving skill identification

Mengxiao Zhu^{a, b}, Li Feng^c, Xin Wang^d, Wei Huang^{e, *}

^a School of Humanities and Social Sciences, University of Science and Technology of China, Hefei, 230026, Anhui, China

^b State Key Laboratory of Cognitive Intelligence, Hefei, 230026, Anhui, China

^c School of Artificial Intelligence and Data Science, University of Science and Technology of China, Hefei, 230026, Anhui, China

^d Institute of Advanced Technology, University of Science and Technology of China, Hefei, 230026, Anhui, China

^e National Education Examinations Authority, Beijing, 100084, China

HIGHLIGHTS

- Proposes a novel framework for automated CPS skill coding.
- Models dependencies via context-aware prompts to pre-trained language models.
- Enhances CPS competence representation by fusing cognitive and social abilities.
- Achieves superior performance on two CPS task datasets over strong baselines.

ARTICLE INFO

Keywords:

Collaborative problem solving
Automated coding
Prompt
Contextual dependency

ABSTRACT

Collaborative Problem Solving (CPS) competence is widely recognized as a critical skill for the 21st century, especially in professions that emphasize communication and teamwork. CPS is a complex process involving both cognitive and social components, and numerous theoretical frameworks have been proposed for its assessment. Currently, CPS competence is typically measured by analyzing process data from simulated tasks. To facilitate a more structured analysis of the CPS process, it is necessary to code behaviors into specific CPS skills. Traditionally, this coding has been conducted manually using predefined coding schemes. However, manual coding is both time-consuming and labor-intensive, making it impractical for large-scale datasets and unsuitable for real-time applications. To overcome these limitations, researchers have developed various automated coding methods. Nonetheless, most existing methods treat each behavior as independent, overlooking the natural dependencies among behaviors in the CPS process. To address this shortcoming, we propose a novel framework called Context-Aware Prompting for CPS (CAP4CPS), which enables automated CPS skill coding by explicitly modeling contextual dependencies. Our approach first employs a context extraction module to retrieve dialogue histories that reflect the cognitive and social aspects of CPS. Next, a context-infused prompting module generates context-rich prompts to guide a pre-trained language model in producing representations of cognitive and social abilities. Finally, an ability fusion module integrates these representations into a unified CPS competency representation, which is then used to predict the corresponding CPS skills. Extensive experiments on two CPS task datasets demonstrate the effectiveness and robustness of the CAP4CPS.

1. Introduction

Collaborative Problem Solving (CPS) refers to the process by which two or more individuals engage in mutual communication to develop a shared solution to a problem (OECD, 2017). Recognized as one of the key competencies of the 21st century (Sun et al., 2020), CPS is

increasingly essential across diverse sectors of modern society. For instance, in software development, effective collaboration is required among front-end engineers, back-end engineers, and product managers. In academia, interdisciplinary research often demands cooperation among scholars with varied expertise. Following natural disasters such

* Corresponding author.

Email addresses: mxzhu@ustc.edu.cn (M. Zhu), fengli@mail.ustc.edu.cn (L. Feng), wangx804@mail.ustc.edu.cn (X. Wang), huangw@mail.neca.edu.cn (W. Huang).

<https://doi.org/10.1016/j.caeai.2026.100567>

Received 31 July 2025; Received in revised form 4 January 2026; Accepted 3 March 2026

Available online 4 March 2026

2666-920X/© 2026 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

as earthquakes or floods, coordinated efforts among governments, rescue organizations, medical institutions, and volunteers are vital for conducting search and rescue operations, distributing supplies, and providing medical care. Given the critical importance of CPS skills, growing attention has been directed toward understanding CPS processes (Tan et al., 2022). A fundamental step in this endeavor is the identification of CPS skills embedded within these processes.

Effective assessment of Collaborative Problem Solving (CPS) first requires the collection of relevant process data. CPS is inherently an interactive, collaborative, and dual-faceted process that involves not only evaluating how individuals engage in logical reasoning to solve problems but also examining how they coordinate social interactions and exchange information through interpersonal communication (OECD, 2017). These complex and dynamic mechanisms pose significant challenges for the accurate and reliable measurement of CPS skills. In real-world contexts, this assessment becomes even more difficult due to the complexity of human-environment collaborative interactions. In recent years, with the rapid advancement of information technology, numerous computer-supported collaborative learning (CSCL) systems (Andrews-Todd et al., 2019; Aslan et al., 2025; Liu et al., 2016) based on the evidence-centered design (ECD) framework (Mislevy et al., 2003) have been developed. These systems typically present participants with collaborative tasks to be completed via computer-mediated interaction. Throughout this process, participants' actions (e.g., mouse clicks or keystrokes) and communications (e.g., messages sent through a chat interface) are logged by the system. These behavioral data can then be analyzed to examine CPS processes. However, because the logged data capture only observable task-related behaviors, they do not directly reveal the underlying CPS skills. Therefore, before conducting in-depth analysis, it is necessary to first code the behavioral data into specific CPS skills.

The coding of behavioral data in CPS processes should be guided by established CPS frameworks. These frameworks define the construct of CPS, delineate its components, and offer a structured representation of the competency. By translating the abstract concept of CPS into observable and measurable dimensions, they enable researchers to systematically analyze participants' interactions and evaluate their collaborative problem-solving abilities. Among these, the Assessment and Teaching of 21st Century Skills (ATC21S) framework and the Programme for International Student Assessment (PISA) framework are two of the most widely adopted models (Hesse et al., 2015; OECD, 2017).

Existing CPS coding methods can be broadly classified into manual and automated approaches. The traditional method relies on human coders to perform annotations (Andrews-Todd and Forsyth, 2020). Manual coding is typically guided by coding schemes (Flor and Andrews-Todd, 2022), which are developed based on theoretical frameworks to map specific behaviors to corresponding CPS skills. Since CPS skills are

manifested through participants' collaborative interactions, coders often consider the surrounding contextual information during the annotation process. Usually, two or more coders are involved, and a calibration phase is conducted to establish high inter-rater reliability and ensure annotation quality. While manual coding can yield high-quality data, it presents several significant limitations. First, it requires extensive training, and coders must carefully examine each data point, making the process highly time-consuming and labor-intensive. As a result, manual coding is impractical for large-scale datasets (Hao et al., 2019). Furthermore, because manual coding is conducted post hoc, it does not support real-time analysis or feedback.

The increasing demand for large-scale data processing and real-time coding has driven growing research interest in the automated coding of CPS skills. Previous studies have proposed various automated coding methods using machine learning (Flor et al., 2016; Stewart et al., 2019), deep learning (Stewart et al., 2021), and pre-trained language models (Pugh et al., 2022; Zhu et al., 2024). However, most of these approaches treat each behavior as an isolated instance, without accounting for interdependencies between behaviors. In practice, CPS is a highly context-dependent process, and the same behavior may correspond to different CPS skills depending on the context in which it occurs. For example, under the PISA framework, the message "What should we do?" may reflect different CPS skills based on the conversational context: if it appears during a discussion of task goals, it is coded as "Discovering the type of collaborative interaction to solve the problem, along with goals"; if it appears during a discussion of action steps, it is coded as "Communicating with team members about the actions to be/being performed." In manual coding, coders consider contextual information to accurately identify the CPS skill represented by each behavior. Although some studies (Hao et al., 2017; Pugh et al., 2021) have attempted to incorporate behavioral dependencies into their models, the methods remain overly simplistic and fail to fully capture the interactive nature of CPS processes. Therefore, further research is needed to develop methods that effectively model contextual dependencies in automated CPS coding.

To address the issues outlined above, this paper proposes a novel method called Context-Aware Prompting for CPS (CAP4CPS), which captures dependencies among behaviors in the CPS process to enable more accurate automated coding. Fig. 1 presents a schematic overview of the CAP4CPS. Specifically, given a chat sequence from a CPS task, we first employ a Context Extraction (CE) Module to independently extract dialogue histories associated with the cognitive and social processes. We then introduce a Context-Infused Prompting (CIP) Module that constructs prompts incorporating the extracted dialogue histories. These prompts allow the pre-trained language model to better interpret the contextual situations in which the chat messages occur and to generate embeddings representing cognitive and social abilities. Subsequently, an

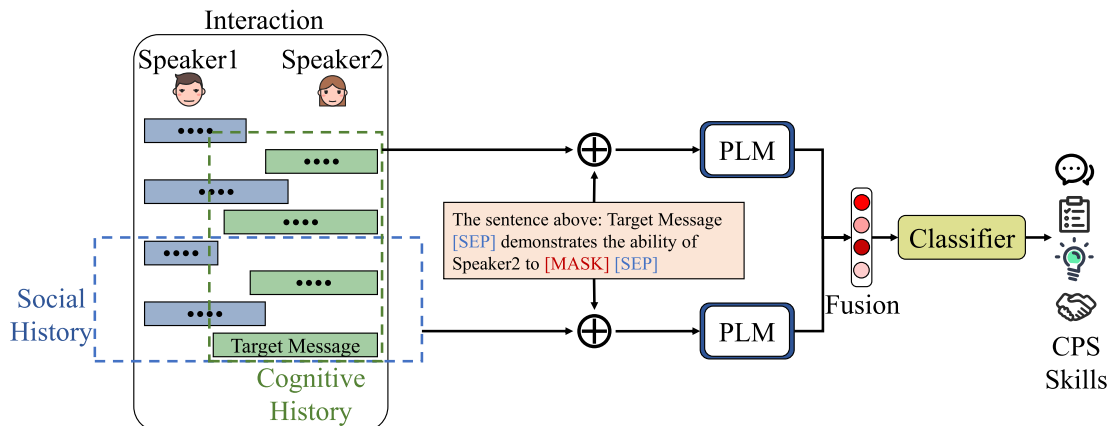


Fig. 1. The schematic diagram of CAP4CPS.

Ability Fusion (AF) Module integrates the cognitive and social embeddings into a unified representation of CPS ability, which is then used to predict CPS skills. Through this design, the proposed model effectively leverages contextual information to capture the specific circumstances surrounding each behavior in the CPS process, thereby improving the accuracy of skill prediction. Finally, we conduct experiments on two datasets to evaluate the effectiveness and robustness of the proposed method for automated CPS coding. In summary, this study makes the following key contributions:

- We propose CAP4CPS, a novel method that captures behavioral dependencies within the CPS process to enable accurate automated coding of CPS skills.
- We explicitly model the interdependencies between cognitive and social abilities to enhance the representation of CPS competence.
- We conduct extensive experiments on two CPS task datasets, demonstrating that our model consistently outperforms existing baselines and validating the effectiveness of our approach for automated CPS coding.

2. Literature review

In this section, we first introduce the concept of CPS frameworks and describe in detail the two frameworks adopted in this study. Next, we review prior research on the automated coding of CPS activities. Finally, we provide a brief overview of the prompt learning paradigm.

2.1. CPS framework

To systematically analyze individuals' behaviors and processes in collaborative problem-solving tasks and to assess their CPS competence, researchers have developed various theoretical frameworks (Andrews-Todd and Kerr, 2019; Hesse et al., 2015; OECD, 2017). These frameworks typically encompass two key dimensions: a social dimension, which pertains to collaboration, and a cognitive dimension, which relates to problem solving (Graesser et al., 2018). In this study, the two constructed datasets are annotated based on two frameworks: the PISA framework (OECD, 2017) and the CPS ontology framework proposed by Andrews-Todd and Kerr (2019), both of which are described in detail below.

The PISA framework (OECD, 2017) categorizes CPS skills into two dimensions—cognitive and social—and provides a detailed classification system. The cognitive dimension comprises four subskills: exploring and understanding, representing and formulating, planning and executing, and monitoring and reflecting. The social dimension includes three subskills: establishing and maintaining shared understanding, taking appropriate actions to solve the problem, and establishing and maintaining team organization. These two dimensions are combined to form a matrix of 12 subskills in total, as shown in Table 1. In this study, the PISA framework is applied to annotate the Mondrian Geometric Fantasy dataset.

Andrews-Todd and Kerr (2019) proposed an ontology framework for CPS research. Similar to the PISA framework, it includes both cognitive and social dimensions but provides a more fine-grained categorization of subskills. The cognitive dimension comprises five subskills: exploring and understanding (CEU), representing and formulating (CRF), planning (CP), executing (CE), and monitoring (CM). The social dimension includes four subskills: maintaining communication (SMC), sharing information (SSI), establishing shared understanding (SESU), and negotiating (SN). Among these, the subskills executing and monitoring appear in both actions and chat messages; therefore, they are further divided into executing actions (CE), executing chats (CEC), monitoring actions (CM), and monitoring chats (CMC), resulting in a total of 11 subskills in the ontology. This framework offers a theoretically grounded structure for mapping specific task behaviors to standardized subskill dimensions. In this study, the ontology framework is employed to annotate the Three-Resistor dataset.

2.2. Automated coding of CPS activities

In recent years, advancements in artificial intelligence have led to the development of various methods for the automated coding of CPS behaviors. These methods can be broadly categorized into context-free and context-aware approaches.

Context-free methods treat each behavior in the CPS process as independent from others. Most existing studies fall into this category. Early research primarily focused on applying traditional machine learning techniques to CPS coding, where feature engineering was a central challenge. For instance, Flor et al. (2016) employed a Naive Bayes classifier with N-gram features and found that using only unigrams produced the best results, while the inclusion of bigrams and trigrams reduced performance. Stewart et al. (2019) used Random Forest classifiers in combination with N-gram and Linguistic Inquiry and Word Count (LIWC) features. Although there was no significant difference in classification accuracy between the two feature sets, LIWC-based coding aligned more closely with human annotations. They also noted that N-gram features pose a risk of overfitting to specific datasets, limiting their generalizability. With the emergence of word embedding models, new approaches have been proposed that utilize semantic representations to improve CPS coding. Stewart et al. (2021) used Global Vectors for Word Representation (GloVe) embeddings in conjunction with Long Short-Term Memory (LSTM) models to capture both short- and long-range word dependencies. Flor and Andrews-Todd (2022) applied a K-Nearest Neighbors (KNN) classifier to predict CPS skills based on semantic similarity, with each utterance represented by the average of its fastText word embeddings. More recently, the rise of pre-trained language models has spurred further advances in CPS coding. Pugh et al. (2022) compared fine-tuned BERT models with Random Forest classifiers and found that both fine-tuned BERT and Random Forest with LIWC features achieved comparable accuracy and generalizability. In contrast, Random Forest models using N-gram features exhibited a tendency to

Table 1
Matrix of collaborative problem solving skills for PISA 2015 (OECD, 2017).

	(1) Establishing and maintaining shared understanding	(2) Taking appropriate action to solve the problem	(3) Establishing and maintaining team organisation
(A) Exploring and understanding	(A1) Discovering perspectives and abilities of team members	(A2) Discovering the type of collaborative interaction to solve the problem, along with goals	(A3) Understanding roles to solve the problem
(B) Representing and formulating	(B1) Building a shared representation and negotiating the meaning of the problem (common ground)	(B2) Identifying and describing tasks to be completed	(B3) Describing roles and team organisation (communication protocol/rules of engagement)
(C) Planning and executing	(C1) Communicating with team members about the actions to be/being performed	(C2) Enacting plans	(C3) Following rules of engagement, (e.g., prompting other team members to perform their tasks)
(D) Monitoring and reflecting	(D1) Monitoring and repairing the shared understanding	(D2) Monitoring results of actions and evaluating success in solving the problem	(D3) Monitoring, providing feedback and adapting the team organisation and roles

overfit and performed poorly on unseen data. [Zhu et al. \(2024\)](#) introduced a prompt learning approach, using manually constructed prompts and answer-label mappings to enable effective CPS skill coding, even with limited training data.

Unlike context-free methods, context-aware approaches consider the dependencies between behaviors in the CPS process. [Flor et al. \(2016\)](#) noted that the CPS label of a sentence in a dialogue may depend on the label of the preceding sentence and experimented with a Hidden Markov Model (HMM) to capture this. Although the overall performance of the HMM was inferior to that of the Naive Bayes classifier, it outperformed Naive Bayes on a specific CPS category—students expressing agreement with teammates—demonstrating that selectively leveraging historical information can improve automated coding. [Hao et al. \(2017\)](#) employed a linear-chain Conditional Random Field (CRF) to model sequential dependencies in CPS dialogues. The CRF's feature functions include transition features, which capture the probabilities of transitions between all label pairs, thus modeling connections between utterances. Their method ultimately outperformed non-sequential models. [Pugh et al. \(2021\)](#) proposed BERT-seq, a method that incorporates both preceding and subsequent utterances when fine-tuning BERT for CPS skill recognition. Specifically, they fine-tuned two separate BERT models to capture forward and backward dependencies, respectively, and obtained the final prediction probability by summing the outputs of the two models.

Given the inherent contextual dependencies in the CPS process, incorporating context into automated coding is essential. While some studies have made preliminary efforts to integrate context, their models primarily address shallow sequential dependencies and remain relatively simplistic. Significant opportunities remain to develop more advanced models that better capture and leverage contextual information.

2.3. Prompt learning

The advent of pre-trained language models (PLMs) has revolutionized the research paradigm in natural language processing (NLP) ([Han et al., 2021](#)). Traditional non-neural methods heavily relied on manually engineered features ([Och et al., 2004](#)). Subsequently, neural network-based approaches gained popularity due to their ability to automatically learn task-specific features from data ([Chung et al., 2014](#); [Hochreiter and Schmidhuber, 1997](#); [Kim, 2014](#)); however, they still required large amounts of labeled task-specific data. The emergence of PLMs ([Devlin et al., 2019](#); [Liu et al., 2019](#); [Radford et al., 2018](#)) transformed this landscape by learning general-purpose language representations through pretraining on large-scale unlabeled corpora, followed by fine-tuning on specific tasks, demonstrating strong performance across diverse downstream applications. BERT (Bidirectional Encoder Representations from Transformers) exemplifies this paradigm shift ([Devlin et al., 2019](#)). It employs a multi-layer bidirectional Transformer encoder architecture and utilizes masked language modeling and next sentence prediction as pretraining objectives. With minimal task-specific parameters and fine-tuning, BERT has achieved remarkable results across numerous NLP tasks. Despite their success, fine-tuning PLMs still requires task-specific datasets, which are often costly and challenging to obtain in large quantities.

In recent years, prompt engineering has gradually emerged as a prominent paradigm in natural language processing (NLP) ([Liu et al., 2023](#)). Unlike traditional fine-tuning, prompt-based methods guide pre-trained language models (PLMs) to perform specific tasks by incorporating textual prompts that describe the task. Prompt learning typically involves three steps: prompt construction, answer retrieval, and answer mapping. For example, in sentiment classification of the sentence “I watched a wonderful movie,” a prompt can be created by appending a task description such as: “I watched a wonderful movie. I felt so _,” where the underscore (“_”) indicates the token the PLM is expected to predict. This prompt is then input into the PLM, which searches for the most

appropriate answer within its vocabulary or a predefined subset. Finally, the predicted answer is mapped to the corresponding label. With carefully designed prompts, PLMs can effectively address tasks in few-shot ([Cui et al., 2022](#)) or even zero-shot ([Zhou et al., 2022](#)) scenarios.

An essential aspect of prompt learning is the construction of effective prompts. Methods for prompt construction can be categorized into manually designed prompts ([Schick and Schütze, 2021a](#)) and automatically generated prompts ([Lester et al., 2021](#)). Manually designed prompts are created based on task characteristics and relevant domain knowledge. In contrast, automatically generated prompts aim to identify suitable prompts for a given task without human intervention. The search space for such prompts may consist of either discrete strings or continuous embedding spaces. Training strategies for prompt learning are classified according to whether the parameters of the prompt and the pre-trained language model (PLM) are updated during training. The main strategies include tuning-free prompting ([Brown et al., 2020](#)), fixed-PLM prompt tuning ([Han et al., 2021](#); [Li and Liang, 2021](#)), fixed-prompt PLM tuning ([Schick and Schütze, 2021a,b](#)), and prompt+PLM tuning ([Ben-David et al., 2022](#)).

Previous experimental results ([Zhu et al., 2024](#)) demonstrate that carefully designed manual prompts improve performance. Therefore, this paper proposes context-integrated manual prompts tailored to the cognitive and social abilities involved in the CPS process. Regarding training strategies, fixed-prompt language model tuning is currently the most widely adopted approach for prompt learning in textual tasks. Our proposed model, CAP4CPS, follows this approach.

3. Dataset

In this paper, we evaluate the performance of CAP4CPS on two datasets to assess its adaptability across different CPS tasks. Therefore, this section introduces two datasets collected from the CPS activities of the Three-Resistor and Mondrian Geometric Fantasy tasks. We provide detailed descriptions of these tasks and outline the data collection and annotation procedures. Finally, we present statistical analyses and visualizations for both datasets.

3.1. Dataset 1: three-resistor

Task Description. The Three-Resistor task ([Horwitz et al., 2017](#)) is a team-based collaboration requiring three participants to work together using physics knowledge. Each participant is responsible for adjusting one resistor. The objective is to ensure that each resistor in the series circuit attains a specific voltage. Because the voltages across the resistors are interdependent, successful task completion demands effective communication and cooperation to ensure all resistors simultaneously reach their target voltages.

In this task, each participant independently operates a computer to simulate an electronic circuit. The task interface, shown in [Fig. 2](#), displays the known conditions and target at the top of the screen. A multimeter for measuring the voltage across the resistor is located at the center. The complete circuit structure appears at the bottom, with a zoom button to view teammates' circuit boards. On the right side, a text chat box facilitates communication among the three participants. A calculator is positioned in the upper right corner to assist with calculations, while the answer submission button is located in the lower right corner.

Four progressively challenging objectives were carefully designed, and participants engaged with them through a level-based format. Teams that advanced through more levels and reached higher task levels within the allotted time were considered to have demonstrated superior performance. [Table 2](#) details the specific settings for the four difficulty levels. Specifically, in Level 1, participants were provided with the external voltage and resistance values, and all teammates shared the same target voltage. Level 2 increased in difficulty by assigning different target voltages to each member. In Level 3, the challenge intensified as

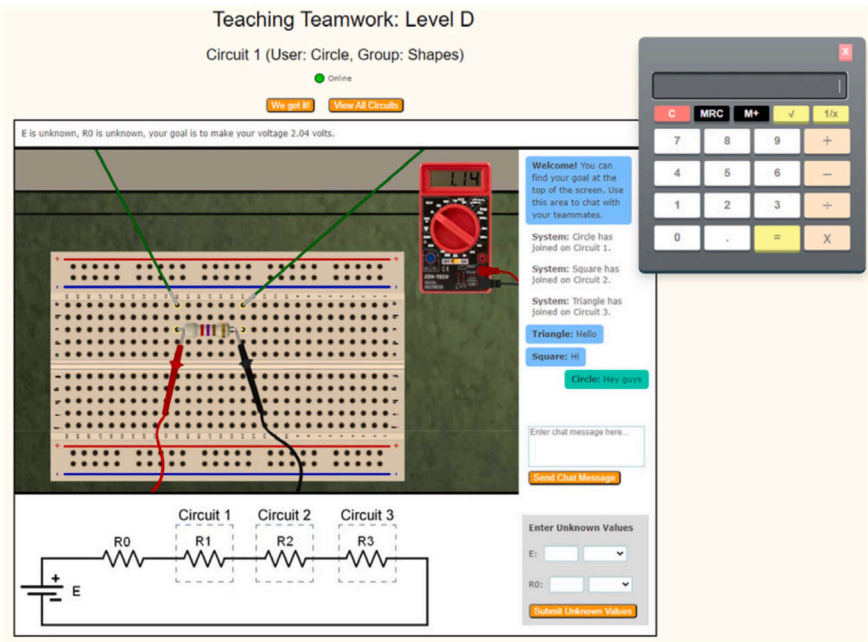


Fig. 2. The operation interface of the Three-Resistor task.

Table 2
The condition settings for different task levels.

Task level	External voltage (E)	External resistance (R0)	Goal voltages
1	Known by all teammates	Known by all teammates	Same for all teammates
2	Known by all teammates	Known by all teammates	Different for all teammates
3	Unknown by teammates	Known by all teammates	Different for all teammates
4	Unknown by teammates	Unknown by teammates	Different for all teammates

teammates were not given the external voltage value, requiring collaboration to infer it. In Level 4, the external resistance value was also withheld from participants.

Dataset Description. The study involved 378 participants, all of whom were students enrolled in electronics, engineering, or physics courses at universities and community colleges across the United States.

Prior to the formal experiment, participants independently completed a questionnaire collecting personal information, including background, race, gender, age, education level, native language, and preference for individual or team work. During the experiment, sessions were conducted in a computer lab where each participant worked independently at a separate computer. Participants were randomly assigned to teams of three, forming a total of 126 teams. The seating arrangement ensured that team members were seated apart. Throughout the task, students remained unaware of their teammates' identities and communicated using virtual usernames (e.g., Lion, Tiger, Bear).

During the task, the system recorded experimental information and students' behavioral data as logs. The experimental information included participant IDs, team names, task details, and other relevant metadata. Behavioral data were categorized into two types: participants' manipulations, such as adjusting resistance or using the calculator, and chat messages from the communication box, including examples like "Did you adjust it?" or "What are your current voltage and resistance readings?". A total of 50,817 data entries were collected, comprising 34,867 manipulation entries and 15,950 chat entries.

The behavioral data collected from participants were coded according to the CPS ontology framework proposed by Andrews-Todd and Kerr (2019). Three trained coders performed the coding following a predefined scheme. To ensure inter-rater reliability, 20% of the dataset was randomly selected and independently coded by all three coders. The overall Cohen's Kappa coefficient was 0.937, with a value of 0.871 specifically for chat entries, indicating a high level of agreement. Subsequently, the remaining data were distributed among the coders for independent annotation.

The final dataset also underwent several preprocessing steps. First, anonymization was performed by removing all personally identifiable information, retaining only group identifiers, usernames, and behavioral data; the usernames used are virtual. The anonymization process is irreversible and does not allow re-identification. In addition, after removing blank chat entries and invalid labels, the final dataset comprised 34,851 manipulation entries and 15,895 chat entries. All behavioral data were encoded into 11 CPS subskills based on the ontology framework. Table 3 presents examples of the manual coding results.

Manipulation data could be directly mapped to CPS subskills, as these actions are discrete and predefined. Specifically, manipulations such as viewing the board, opening zoom, and submitting results were categorized as monitoring (CM); using a calculator was classified as executing (CE); and changing the resistor value was coded based on timing—before planning, it was categorized as exploring and understanding (CEU), and after planning, as executing (CE). Thus, manipulation data could be readily identified and accurately mapped to CPS subskills.

Fig. 3(a) (left panel) presents the distribution of CPS subskills in chat data, encompassing eight distinct subskills. Among these, sharing information (SSI) has the highest proportion, followed by establishing shared understanding (SESU); together, they account for nearly 60% of all instances. Representing and formulating (CRF) appears the least frequently, comprising only 2.2%. In terms of broader dimensions, the social dimension constitutes 75.1% of the data, significantly exceeding the cognitive dimension. Overall, the distribution reveals notable imbalances both across subskills and between the two dimensions, which presents challenges for automated coding.

Table 3
Examples of the data collected from the Three-Resistor task and its manual coding results.

Data type	Dimension	Code	Subskill	Frequency	Example
Manipulation	Cognitive	CM	Monitoring actions	5970	Viewing board; opening zoom; submitting results
	Cognitive	CE	Executing actions	23,572	Changing resistor value; using calculator
	Cognitive	CEU	Exploring and understanding	5309	Changing resistor value prior to developing a plan
Chat message	Cognitive	CRF	Representing and formulating	356	"I think we still have to get to the correct voltage of each resistor."
	Cognitive	CP	Planning	1066	"Everyone try to make all of your resistors equal to 100 ohms."
	Cognitive	CMC	Monitoring communication	1193	"It keeps telling me mine's incorrect."
	Cognitive	CEC	Executing communication	1348	"I got 325, I changed it to 330."
	Social	SESU	Establishing shared understanding	3317	"What if everyone sets there resistor to, let's say, 1k Ohms?"
	Social	SMC	Maintaining communication	1289	"It's cool."
	Social	SN	Negotiating	1149	"I think so."
	Social	SSI	Sharing information	6177	"I currently have 2.0 volts with a 180 ohm resistor."

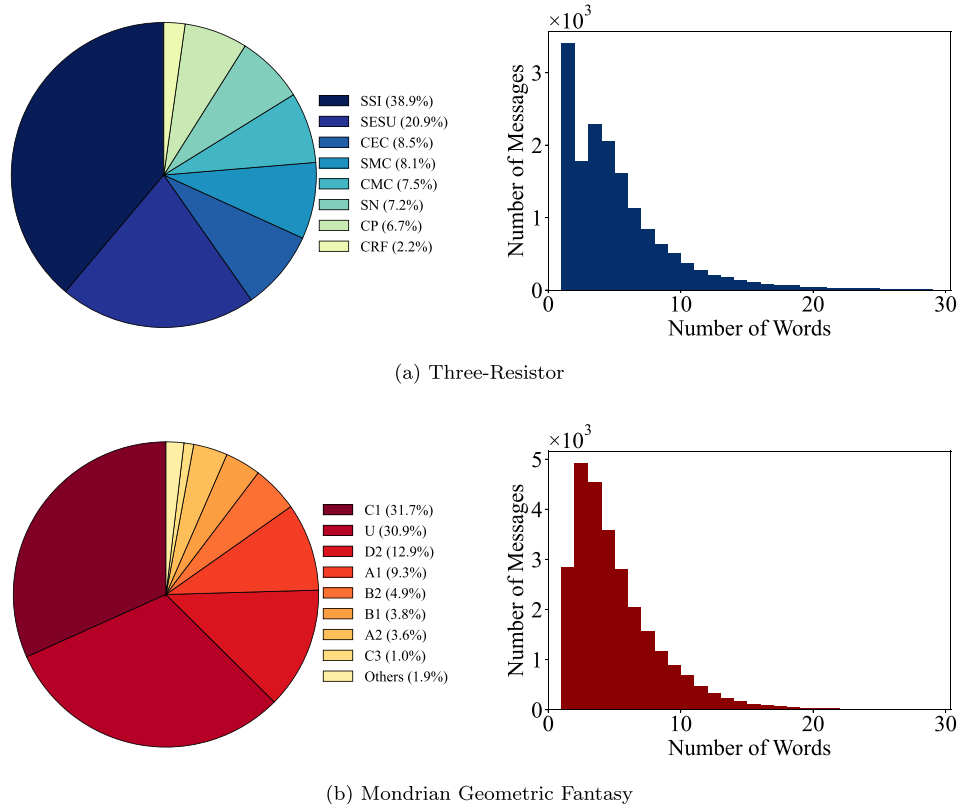


Fig. 3. The CPS subskill proportions and chat message word count distribution of the Three-Resistor and Mondrian Geometric Fantasy datasets.

Fig. 3(a) (right panel) illustrates the distribution of chat entry lengths based on word count. The data show that most chat entry are brief: 99% contain fewer than 20 words.

3.2. Dataset 2: mondrian geometric fantasy

Task Description. The Mondrian Geometric Fantasy task (Xu et al., 2024) is framed within a fictional narrative: two paintings by the renowned artist Mondrian, housed in a museum collection, have been shattered by a thief. At the request of the museum director, Afu, three students each recover a portion of the fragmented artwork. Together, they must collaborate to restore the paintings to their original form.

The Mondrian Geometric Fantasy task is a collaborative activity completed by three participants, who must translate and rotate fragments to reconstruct the missing areas of a canvas within a specified time limit. The task consists of two levels, with the second level including more fragments than the first. Participants can only advance to the second level after successfully submitting the first-level result

and receiving confirmation. Teams that assemble more fragments receive higher scores. During the task, each participant possesses different fragments and sees different shadowed and blank areas. While each participant has structural information about where others' fragments should be placed, they lack information about the correct placement of their own fragments. Therefore, successful completion of the task requires effective communication and coordination among the three participants to fully restore the paintings.

The interface of the online task is illustrated in Fig. 4. The central area of the screen is the task workspace, where a shattered Mondrian painting is displayed, consisting of shadowed and blank regions. The shadowed regions indicate where teammates' fragments should be placed and are divided by white boundary lines into numbered zones. Fragments must be moved into their corresponding zones without crossing these boundaries. The white regions represent the areas designated for the current participant's fragments. However, the participant cannot see the specific division of these regions; instead, they appear as shadowed areas to the other teammates, who have access to this layout information. Fragments,

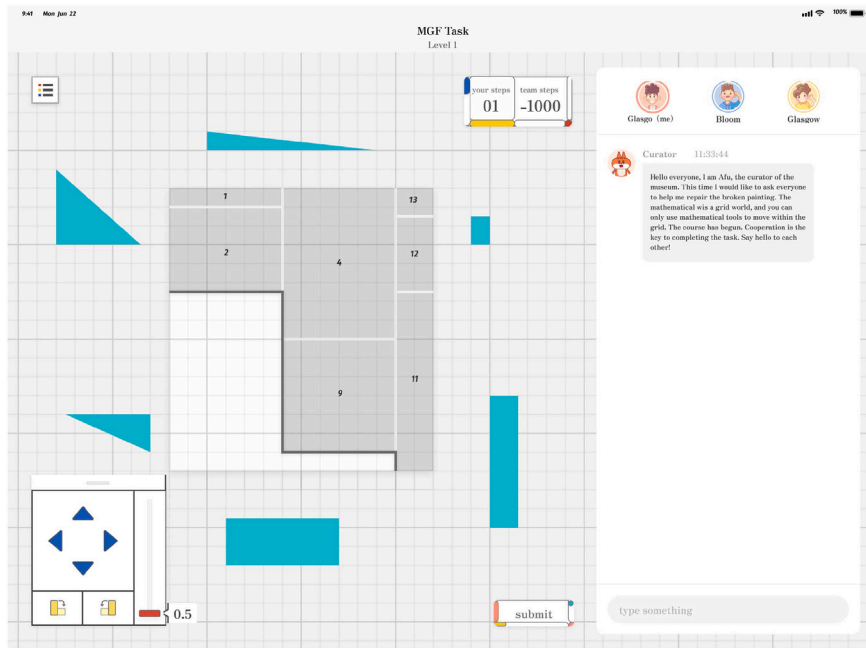


Fig. 4. The operation interface of the Mondrian Geometric Fantasy task.

shaped as triangles and rectangles of various sizes, are scattered around the canvas, with each participant having a unique set. Thus, successful task completion requires team members to exchange information effectively. In the bottom-left corner of the screen is the control panel, where participants can click buttons to translate and rotate fragments. The translation step size can be adjusted, influencing the total number of operations required. On the right side of the screen is the chat panel, which allows participants to communicate via real-time text messaging. The top-right corner displays a countdown timer and a step counter to help participants track their progress. A submit button is located in the bottom-right corner and can be used when participants believe the task is complete. If the submission fails, they have the option to restart the task or exit.

Dataset Description. The data were collected from 840 students in Grades 10 and 11 at a high school in China. The task was administered in the school's computer lab, with each participant using a computer independently. Participation was anonymous, and participants were unaware of each other's real identities.

The overall experimental procedure consisted of six stages: logging into the system interface, automated team assignment, clicking "Start," watching a tutorial animation, completing the online CPS task, and filling out a post-task questionnaire. Teams were formed randomly, with three students per team. Before beginning the task, participants watched a tutorial animation lasting approximately 90 s, which explained the task rules and operational instructions. The task was limited to 40 minutes, while the post-task questionnaire had no time constraint. Throughout the task, all manipulation and chat data were automatically recorded by the system.

A total of 280 teams participated in the experiment. Due to issues such as unexpected disconnections and missing data, 51 teams were excluded as invalid samples. Ultimately, 229 valid team samples remained, comprising a total of 245,034 data entries—216,547 manipulation entries and 28,487 chat entries.

The coding was conducted by two coders following the guidelines established under the PISA 2015 framework. All manipulation entries were coded as enacting plans (C2), while the chat entries contained no instances of C2 skills. Chat entries that could not be matched to any competencies defined in the PISA framework (OECD, 2017) were labeled

as unknown (U), indicating an unspecified competency. Initially, the two coders collaboratively coded four randomly selected teams through face-to-face discussions to align their understanding of the coding criteria. They then independently coded another five randomly selected teams. The resulting Cohen's Kappa coefficient was 0.827, indicating a high level of inter-rater agreement. The remaining data were subsequently divided between the two coders for independent coding.

The final dataset likewise underwent the same anonymization procedure as the Three Resistor dataset. In addition, the final dataset included only chat entries from the formal task phase, with blank chat entries removed. This resulted in 216,547 manipulation entries and 26,715 chat entries. All chat entries were coded into 12 CPS subskills. Table 4 presents examples of the manual coding results.

All manipulation entries were categorized under Enacting Plans (C2). The following discussion focuses primarily on the CPS subskills corresponding to chat entries. As shown on the left side of Fig. 3(b), the most prevalent skill is Communicating with team members about the actions to be/being performed (C1), accounting for 31.7%. In addition, two other skills also exhibit relatively high proportions: monitoring results of actions and evaluating success in solving the problem (D2), which accounts for 12.9%, and discovering perspectives and abilities of team members (A1), which accounts for 9.3%. Together, these three most frequent skills, along with the unknown (U) subskill, comprise 84.8% of the dataset. All remaining subskills each account for less than 5%, with some contributing even less than 1%. Overall, the distribution of subskills in chat entries is highly imbalanced, posing challenges for automated coding.

The image on the right side of Fig. 3(b) illustrates the word count distribution of chat entries. Similar to the Three-Resistor dataset, most chat entries are brief—99% contain fewer than 16 words.

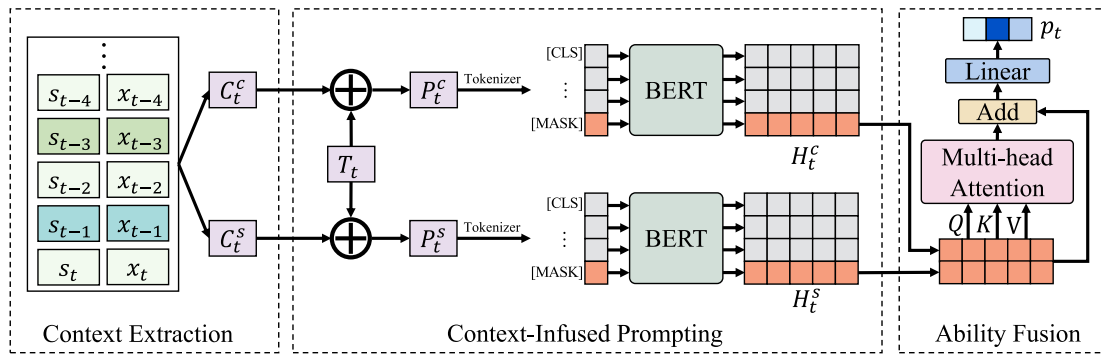
4. Preliminaries

Based on the discussion in the previous section, it is clear that manipulation data in the CPS automated coding task can be reliably coded using rule-based matching. Therefore, this study focuses on the automated coding of chat data within the CPS task. Specifically, the CPS coding problem can be formulated as follows.

Table 4

Examples of the data collected from the Mondrian Geometric Fantasy task and its manual coding results.

Data type	Code	Subskill	Frequency	Example
Manipulation	C2	Enacting plans	216,547	Select fragments; move fragments; rotate fragments; change steps; submitting results
Chat message	A1	Discovering perspectives and abilities of team members	2487	"I've got a triangle that's 6 long and 8 high."
	A2	Discovering the type of collaborative interaction to solve the problem, along with goals	967	"Did you understand the rules?"
	A3	Understanding roles to solve the problem	63	"I'll take care of the bottom left corner."
	B1	Building a shared representation and negotiating the meaning of the problem (common ground)	1011	"Take the top right corner as the origin."
	B2	Identifying and describing tasks to be completed	1313	"Just fill in the shaded area."
	B3	Describing roles and team organisation (communication protocol/rules of engagement)	222	"Follow my lead."
	C1	Communicating with team members about the actions to be/being performed	8470	"Move Blue A and Red A upwards to the area 6."
	C3	Following rules of engagement, (e.g., prompting other team members to perform their tasks)	275	"Hurry up!"
	D1	Monitoring and repairing the shared understanding	89	"Fragments cannot be placed across regions."
	D2	Monitoring results of actions and evaluating success in solving the problem	3445	"There is a gap above my yellow B triangle."
	D3	Monitoring, providing feedback and adapting the team organisation and roles	130	"Why isn't the other person talking?"
	U	Unknown	8243	–

**Fig. 5.** The architecture of CAP4CPS.

Definition 4.1 (CPS Coding Problem). Given a team of messages D from one CPS task and their corresponding CPS skills Y , the goal of CPS coding is to find a model f that establishes a mapping between the messages D and the CPS skills Y :

$$f(D; \theta) = Y, \quad (1)$$

where θ represents the parameters of the model f . $D = \{(s_i, x_i)\}_{i=1}^N$ is a dialogue, where x_i represents the i -th message and s_i represents the corresponding speaker of x_i , N represents the number of messages. $Y = \{y_i\}_{i=1}^N$ are labels of all messages, where y_i represents the label of x_i . And $y_i \in C$, where C is the set of all CPS skills.

5. Method

This section presents the technical details of Context-Aware Prompting for CPS (CAP4CPS) used to identify CPS skills. As illustrated in Fig. 5, CAP4CPS comprises three main components: the Context Extraction (CE) Module, the Context-Infused Prompting (CIP) Module, and the Ability Fusion (AF) Module. Specifically, CE extracts context information for the cognitive and social dimensions, respectively. Next, CIP generates representations that capture dependencies between these contexts by prompting pre-trained language models. Finally, AF integrates competencies from both dimensions, and a classifier predicts the CPS skill associated with each message.

5.1. Context extraction module

Before constructing the prompts, we propose the Context Extraction (CE) module to separately extract contextual information from the cognitive and social dimensions, facilitating the identification of CPS skills

reflected in messages. Given that in real-time communication environments participants' CPS skills depend on prior dialogue, we use only historical dialogue as context.

Contextual information plays a critical role in encoding CPS skills during the CPS process. The same message may correspond to different CPS skills depending on its context. For instance, in the Mondrian Geometric Fantasy task, a participant's message, "What should we do?", should be encoded as "Discovering the type of collaborative interaction to solve the problem, along with goals (A2)" if it occurs during the goal-discussion phase. However, if the message arises during the discussion of how to move specific fragments, it should be encoded as "Communicating with team members about the actions to be/being performed (C1)." Thus, selecting appropriate contextual information to clarify the CPS phase in which a message occurs is essential.

Understanding situations related to the cognitive and social dimensions requires distinct types of contextual information. Collaborative problem solving is a complex process involving both the cognitive dimension of individual problem solving and the social dimension of collaborative efforts. In the cognitive dimension, problem solving competency is defined as "an individual's capacity to engage in cognitive processing to understand and resolve problem situations where a method of solution is not immediately obvious." Therefore, understanding the cognitive context relies on an individual's personal historical dialogue. In contrast, in the social dimension, collaborative problem solving competency is defined as "the capacity of an individual to effectively engage in a process whereby two or more agents attempt to solve a problem by sharing the understanding and effort required to come to a solution, and pooling their knowledge, skills, and effort to reach that solution." Consequently, understanding the social context requires interaction history with others.

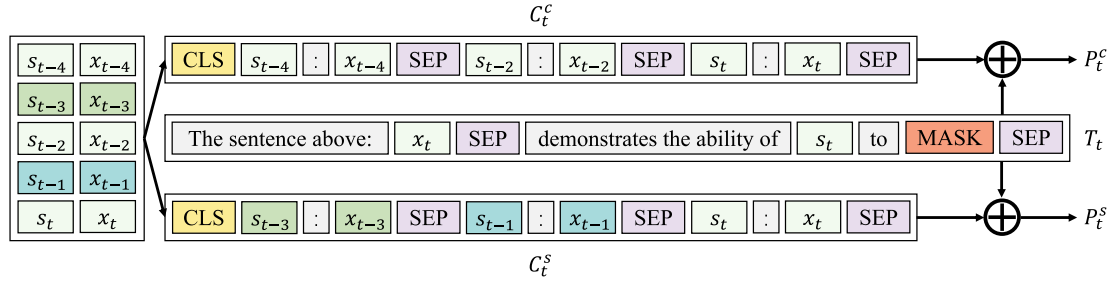


Fig. 6. Details of constructing prompts that incorporate context.

Next, we introduce how the contextual information for the cognitive and social dimensions is extracted, respectively. Fig. 6 illustrates this process.

Cognitive Context is derived from an individual's prior dialogue to understand the problem-solving process from the perspective of a single participant. As illustrated in Fig. 5, let the target message to be classified be denoted as x_t . For the message x_t sent by speaker s_t , the index set of the speaker's historical dialogue is defined as:

$$J_t^c = \{i \mid s_i = s_t, i \leq t\}, \quad (2)$$

which can be sorted in descending order as:

$$J_t^c = \{i_0^c, i_1^c, i_2^c, \dots\}, \quad (3)$$

where $i_0^c > i_1^c > i_2^c > \dots$ and $i_0^c = t$. The cognitive context is composed of the most recent w^c messages, whose indices are $i_1^c, i_2^c, \dots, i_{w^c}^c$. These messages, together with their corresponding speakers, are sequentially concatenated as follows:

$$C_t^c = [\text{CLS}]s_{i_{w^c}^c} : x_{i_{w^c}^c} [\text{SEP}]s_{i_{w^c-1}^c} : x_{i_{w^c-1}^c} [\text{SEP}] \dots [\text{SEP}]s_{i_1^c} : x_{i_1^c} [\text{SEP}], \quad (4)$$

where C_t^c represents the cognitive context corresponding to the t -th message, and w^c denotes the window size for the historical dialogue within the cognitive context. The tokens [CLS] and [SEP] are special markers used in the pre-trained language model BERT, where [CLS] is typically placed at the beginning of the input sequence, and [SEP] is used to separate different sentences or segments of text.

Social Context is extracted from the historical dialogue involving other participants. For the message x_t , the index set of prior dialogue from other participants is defined as:

$$J_t^s = \{i \mid s_i \neq s_t, i \leq t\}, \quad (5)$$

which can be sorted in descending order as:

$$J_t^s = \{i_1^s, i_2^s, i_3^s, \dots\}, \quad (6)$$

where $i_1^s > i_2^s > i_3^s > \dots$. The social context consists of the most recent w^s messages from other participants, whose indices are $i_1^s, i_2^s, \dots, i_{w^s}^s$. These messages, along with their corresponding speakers, are sequentially concatenated as follows:

$$C_t^s = [\text{CLS}]s_{i_{w^s}^s} : x_{i_{w^s}^s} [\text{SEP}]s_{i_{w^s-1}^s} : x_{i_{w^s-1}^s} [\text{SEP}] \dots [\text{SEP}]s_{i_1^s} : x_{i_1^s} [\text{SEP}], \quad (7)$$

where C_t^s represents the social context corresponding to the t -th message, and w^s denotes the window size for the historical dialogue within the social context.

5.2. Context-infused prompting module

After extracting the cognitive and social contexts separately, we propose Context-Infused Prompting (CIP) module to construct prompts that

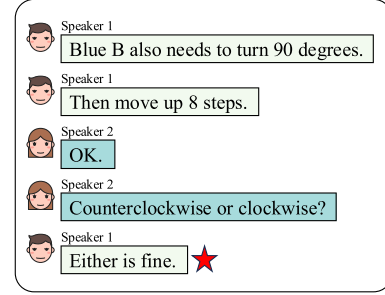


Fig. 7. An example of a dialogue. The message marked with a red star is the target message to be classified. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

infused contextual information. Inspired by Song et al. (2022), we concatenate the context with the prompt. These prompts are then used with the pre-trained language model BERT to obtain embeddings for cognitive and social abilities. Specifically, for a chat message x_t sent by speaker s_t , we design the following prompt template:

$$T_t = \text{The sentence above: } x_t [\text{SEP}] \text{ demonstrates the ability of } s_t \text{ to } [\text{MASK}] [\text{SEP}], \quad (8)$$

where [MASK] is the special token to be predicted by BERT. We then concatenate the cognitive and social contexts with the prompt template to form the final prompts:

$$P_t^c = C_t^c \oplus T_t, \quad (9)$$

$$P_t^s = C_t^s \oplus T_t, \quad (10)$$

where P_t^c and P_t^s represent the prompts for the cognitive and social dimensions, respectively, and $\oplus$ denotes the concatenation operation.

Illustrative Example. To more clearly illustrate the final prompt forms constructed for the cognitive and social dimensions, we present two concrete examples. Fig. 7 shows a fragment of a dialogue from the Mondrian Geometric Fantasy dataset. We take Speaker 1's utterance "Either is fine" as the target message. If the window size of the cognitive context is set to $w^c = 2$ and that of the social context is set to $w^s = 1$, then according to the context extraction method described above, we extract two historical utterances from Speaker 1 and one historical utterance from other speakers. Consequently, the final prompts constructed for the cognitive and social dimensions are as follows:

$$P_t^c = [\text{CLS}] \text{ Speaker 1: Blue B also needs to turn 90 degrees. } [\text{SEP}] \text{ Speaker 1: Then move up 8 steps. } [\text{SEP}] \text{ Speaker 1: Either is fine. } [\text{SEP}] \text{ The sentence above: Either is fine. } [\text{SEP}] \text{ demonstrates the ability of Speaker 1 to } [\text{MASK}] [\text{SEP}] \quad (11)$$

$$P_t^s = [\text{CLS}] \text{ Speaker 2: Counterclockwise or clockwise? } [\text{SEP}] \text{ Speaker 1: Either is fine. } [\text{SEP}] \text{ The sentence above: Either is fine. } [\text{SEP}] \text{ demonstrates the ability of Speaker 1 to } [\text{MASK}] [\text{SEP}] \quad (12)$$

After obtaining the prompts for both dimensions, we feed them into BERT to get the embedding representations of the [MASK] token:

$$H_t^c = \text{BERT}(P_t^c), \quad (13)$$

$$H_t^s = \text{BERT}(P_t^s), \quad (14)$$

where $H_t^c, H_t^s \in \mathbb{R}^{l \times d}$ represent the output embeddings from BERT, l is the sequence length, and d is the embedding dimension. Finally, we extract the embeddings at the [MASK] token position to obtain the cognitive ability representation $E_t^c \in \mathbb{R}^d$ and the social ability representation $E_t^s \in \mathbb{R}^d$.

5.3. Ability fusion module

After obtaining the representations of cognitive and social abilities, we propose the Ability Fusion (AF) module to integrate them into a unified CPS ability representation while capturing their relative contributions to different CPS skills. In CPS processes, cognitive and social abilities are interdependent and mutually influential, and their relative importance varies across skills. When applying CPS coding to specific task scenarios, the emphasis on cognitive or social abilities often depends on the nature of the task. To address this, we employ an attention mechanism to automatically learn the contribution weights of cognitive and social abilities and fuse them into a unified CPS ability representation for skill prediction.

Specifically, we first represent the cognitive ability as E_t^c and the social ability as E_t^s , and concatenate them into a sequence $[E_t^{c\top}; E_t^{s\top}] \in \mathbb{R}^{2 \times d}$. This sequence is then linearly projected to obtain the queries, keys, and values for H attention heads:

$$Q_h = [E_t^{c\top}; E_t^{s\top}]W_h^Q, \quad K_h = [E_t^{c\top}; E_t^{s\top}]W_h^K, \quad V_h = [E_t^{c\top}; E_t^{s\top}]W_h^V, \quad (15)$$

where $W_h^Q, W_h^K, W_h^V \in \mathbb{R}^{d \times d_k}$ are learnable projection matrices for the h -th attention head, which can be interpreted as linear layer without bias. Here, $h \in [1, H]$ denotes the index of the attention head, H is the total number of attention heads, and $d_k = d/H$. Then, for each attention head, we perform scaled dot-product attention:

$$\text{Attention}_h(Q_h, K_h, V_h) = \text{softmax}\left(\frac{Q_h K_h^\top}{\sqrt{d_k}}\right) V_h, \quad (16)$$

where the $\text{softmax}(\cdot)$ function ensures that the weights assigned to the cognitive and social abilities sum to 1.

Finally, we concatenate the outputs of all attention heads and pass the result through a linear layer to obtain the fused CPS ability representation:

$$[A_t^{c\top}; A_t^{s\top}] = [\text{head}_1, \dots, \text{head}_H]W^O, \quad (17)$$

where $\text{head}_h = \text{Attention}_h(Q_h, K_h, V_h)$, and $W^O \in \mathbb{R}^{d \times d}$ is a learnable projection matrix. $A_t^c, A_t^s \in \mathbb{R}^d$ are the outputs of the multi-head attention. Finally, these two components are concatenated, and a residual connection is applied to obtain the CPS ability representation.

$$E_t^f = [A_t^c; A_t^s] + [E_t^c; E_t^s], \quad (18)$$

where $E_t^f \in \mathbb{R}^{2d}$ denotes the representation of CPS ability.

Through the Ability Fusion (AF) module, we obtain the fused CPS ability representation E_t^f , which integrates both cognitive and social abilities.

5.4. CPS skill prediction

After obtaining the final representation of the CPS ability, we use it to predict the CPS skill corresponding to the message m_i . Specifically,

we apply a linear layer:

$$p_i = \sigma(W^f E_t^f + b^f), \quad (19)$$

where $W^f \in \mathbb{R}^{|C| \times 2d}$ is a randomly initialized weight matrix, $b^f \in \mathbb{R}^{|C|}$ is a bias vector, and $\sigma(\cdot)$ is a nonlinear activation function. $p_i \in \mathbb{R}^{|C|}$ represents the predicted probabilities over the C .

The loss function is the cross-entropy loss:

$$\mathcal{L} = - \sum_{i=1}^N \sum_{c=1}^{|C|} y_{i,c} \cdot \log(p_{i,c}), \quad (20)$$

where $y_{i,c}$ denotes the c -th element of the vector y_i , $p_{i,c}$ denotes the c -th element of the vector p_i .

6. Experiments

In this section, we conduct comprehensive experiments on the Three-Resistor and Mondrian Geometric Fantasy datasets to evaluate the effectiveness of the CAP4CPS model in comparison with baseline models. Our objective is to address the following research questions:

- **RQ1:** How does CAP4CPS perform in predicting CPS skills compared to baseline models?
- **RQ2:** What is the impact of each module within CAP4CPS on overall model performance?
- **RQ3:** How do the cognitive context window w^c and the social context window w^s influence the model's performance?

6.1. Experimental setup

Pre-trained Language Model. We use BERT as the pre-trained language model in all our experiments. Specifically, we employ the `bert-base-cased` model for the Three-Resistor dataset and the `bert-base-chinese` model for the Mondrian Geometric Fantasy dataset. The choice of pre-trained models is guided by the primary language of each dataset to ensure optimal semantic representation.

CAP4CPS Setting. We employ grid search to find the best hyperparameters using the validation set. Specifically, we adjust the cognitive context window size (w^c) and the social context window size (w^s) within the range $\{1, 2, 3\}$, the number of attention heads (h) within the range $\{1, 2, 4, 8\}$. Finally, we set the cognitive context window size (w^c) to 2, the social context window size (w^s) to 1, and the number of attention heads (h) to 2. There are also several other parameters. The embedding dimension (d) for both cognitive and social abilities is set to 768. The maximum token length (l) of the final prompt is set to 140 for the Three-Resistor dataset and 112 for the Mondrian Geometric Fantasy dataset, based on the distribution of message word counts shown in Fig. 3.

Training Details. For dataset preparation, we first apply stratified sampling to split the data into training, validation, and test sets with a ratio of 7:1.5:1.5, ensuring that the data distribution is consistent with the original dataset. During training, we use the AdamW optimizer with a learning rate of 2×10^{-5} and weight decay of 1×10^{-2} . The size of mini-batch is set to 32. A dropout rate of 0.1 is applied to prevent overfitting. In addition, we adopt an early stopping strategy, where training is terminated if the validation loss does not decrease for two consecutive epochs.

6.2. Baseline

To demonstrate the effectiveness of the CAP4CPS model, we compare it with baseline models used in previous CPS coding studies, as well as other commonly employed text classification models. Based on whether these models utilize context during automated coding, they are categorized as follows:

Context-free model

- **Naive Bayes** (Flor et al., 2016), **Random Forest** (Stewart et al., 2019), **KNN** (Flor and Andrews-Todd, 2022): For these models, we use TF-IDF features. When constructing TF-IDF representations, the vocabulary is limited to the top 10,000 most frequent n-grams, and unigrams, bigrams, and trigrams are extracted. In addition, L2 normalization is applied to each document vector and IDF smoothing is enabled. Furthermore, since TF-IDF has weaker representational capacity than BERT mean-pooled sentence embeddings, we additionally evaluate Random Forest and KNN using BERT mean-pooled sentence embeddings as sentence-level features, in order to more fairly compare the impact of model architecture on coding performance. Because Naive Bayes does not support features with negative values, we do not incorporate BERT mean-pooled embeddings for this model. These models use feature dimensions equal to the TF-IDF vocabulary size when TF-IDF features are adopted, and a feature dimension of 768 when mean-pooled BERT sentence embeddings are used as features.
- **CNN** (Kim, 2014), **GRU** (Chung et al., 2014), **LSTM** (Stewart et al., 2021): These approaches leverage deep learning techniques for feature extraction, followed by a linear layer for classification. For these models, we construct a vocabulary consisting of all words with a frequency greater than 2. Since the architectural differences of these models are reflected in the construction of sentence-level embeddings, we do not additionally include experiments using BERT mean-pooled sentence embeddings. The embedding dimension of all these models is set to 300. The hidden state dimension of both the LSTM and GRU is 256. The CNN employs convolutional filters with kernel sizes of 3, 4, and 5, each with 256 output channels.
- **BERT-finetune** (Pugh et al., 2022), **BERT-prompt** (Zhu et al., 2024): Both methods are based on pre-trained language models. BERT-finetune performs classification by fine-tuning BERT with an additional linear layer, while BERT-prompt employs a prompt-based approach that enables BERT to generate an output subsequently mapped to the appropriate classification label. The hidden state dimension of all these models is set to BERT's default size of 768.

Context-aware model

- **CPS-Rater** (Hao et al., 2017): CPS-Rater utilizes a linear-chain Conditional Random Field (CRF) to explicitly model transition probabilities between adjacent labels, thereby enhancing classification performance. In the feature construction stage, we build vocabularies of unigrams, bigrams, and trigrams as state features. In addition, we incorporate BERT mean-pooled sentence embeddings as supplementary features.
- **BERT-seq** (Pugh et al., 2021): BERT-seq models dependencies between the current message and its preceding and following messages separately, using the embedding representation of a separator token to facilitate classification.

To clearly illustrate the differences among the models, Table 5 summarizes their characteristics. Machine learning methods were implemented using scikit-learn, deep learning methods were implemented using PyTorch. For the above models that rely on n-gram-based vocabularies, the Three-Resistor dataset uses the default whitespace tokenizer, whereas Mondrian Geometric Fantasy adopts jieba¹ tokenizer. In contrast, all experiments involving BERT employ BERT's default tokenizer. Experiments were conducted on a Linux server equipped with two 2.0 GHz Intel Xeon 8358 CPUs and an RTX 4090 GPU. To ensure a fair comparison, all baseline parameters were carefully optimized for optimal performance.

¹ <https://github.com/fxsjy/jieba>

Table 5

Characteristics of all models.

Model	PLM	Context	Multi-dimension
Naive bayes	–	–	–
Random forest	–	–	–
Random forest (BERT)	✓	–	–
KNN	–	–	–
KNN (BERT)	✓	–	–
CNN	–	–	–
GRU	–	–	–
LSTM	–	–	–
BERT-finetune	✓	–	–
BERT-prompt	✓	–	–
CPS-Rater	–	✓	–
CPS-Rater (BERT)	✓	✓	–
BERT-seq	✓	✓	–
CAP4CPS	✓	✓	✓

6.3. Evaluation metrics

We adopt the following evaluation metrics commonly used in CPS coding research:

Accuracy. Accuracy is a widely used metric in classification tasks, defined as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad (21)$$

where TP , TN , FP , and FN denote the numbers of true positives, true negatives, false positives, and false negatives, respectively.

Weighted F1 Score. The Weighted F1 Score (W-F1) is the weighted average of the F1 scores of all classes, weighted by the number of samples in each class. This metric effectively reflects overall performance under class imbalance. It is defined as follows:

$$\text{Weighted F1} = \sum_{i=1}^K \frac{n_i}{N} \cdot F1_i \quad (22)$$

where K denotes the number of classes, n_i represents the number of samples in class i , N is the total number of samples, and $F1_i$ is the F1 score of class i .

Cohen's Kappa coefficient. Cohen's Kappa coefficient κ is a statistical measure that evaluates the agreement between two raters, which adjusts for the possibility that raters might agree purely by random chance. $\kappa \in [-1, 1]$, where $\kappa = 1$ indicates perfect agreement, $\kappa = 0$ indicates agreement equivalent to random chance, and $\kappa < 0$ indicates less agreement than would be expected by chance. It is defined as:

$$\kappa = \frac{p_o - p_e}{1 - p_e}, \quad (23)$$

$$p_o = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(y_i = \hat{y}_i), \quad (24)$$

$$p_e = \sum_{i=1}^K \frac{n_i \times \hat{n}_i}{N^2}, \quad (25)$$

where $\mathbb{I}(A)$ is the indicator function, which equals 1 if A is true and 0 otherwise. y_i denotes the true label, and $\hat{y}_i$ denotes the predicted label. n_i represents the number of samples belonging to class i in the true labels, and $\hat{n}_i$ represents the number of samples predicted as class i .

6.4. Experimental results

6.4.1. Performance comparison (RQ1)

To demonstrate the practical significance of our proposed model, we compared CAP4CPS with all baseline methods on the Three-Resistor and Mondrian Geometric Fantasy datasets. Each experiment was repeated

Table 6Performance comparison on CPS annotation tasks in terms of accuracy, W-F1 and Kappa ($\times 100$).

Model	Three-Resistor			Mondrian Geometric Fantasy		
	Accuracy	W-F1	Kappa	Accuracy	W-F1	Kappa
Naive bayes	63.58 $\pm$ 0.58	59.54 $\pm$ 0.66	48.14 $\pm$ 0.89	70.00 $\pm$ 0.56	67.59 $\pm$ 0.65	59.40 $\pm$ 0.78
Random forest	69.29 $\pm$ 0.53	68.66 $\pm$ 0.54	59.35 $\pm$ 0.67	77.57 $\pm$ 0.68	76.85 $\pm$ 0.70	70.37 $\pm$ 0.90
Random forest (BERT)	67.01 $\pm$ 0.87	63.99 $\pm$ 0.94	54.09 $\pm$ 1.23	76.00 $\pm$ 0.61	74.89 $\pm$ 0.62	67.89 $\pm$ 0.85
KNN	55.51 $\pm$ 0.72	54.62 $\pm$ 0.92	40.50 $\pm$ 1.45	66.15 $\pm$ 1.12	65.22 $\pm$ 1.06	54.64 $\pm$ 1.52
KNN (BERT)	68.90 $\pm$ 0.88	67.72 $\pm$ 0.90	58.50 $\pm$ 1.17	77.88 $\pm$ 0.55	77.78 $\pm$ 0.53	71.37 $\pm$ 0.69
CNN	69.90 $\pm$ 1.06	69.36 $\pm$ 1.27	60.39 $\pm$ 1.64	77.43 $\pm$ 0.80	77.01 $\pm$ 0.82	70.51 $\pm$ 1.03
GRU	72.96 $\pm$ 0.67	72.80 $\pm$ 0.67	64.84 $\pm$ 0.87	78.67 $\pm$ 0.71	78.35 $\pm$ 0.72	72.20 $\pm$ 0.94
LSTM	72.55 $\pm$ 0.98	72.33 $\pm$ 0.94	64.22 $\pm$ 1.24	78.77 $\pm$ 0.50	78.47 $\pm$ 0.53	72.33 $\pm$ 0.67
BERT-finetune	80.24 $\pm$ 0.49	80.13 $\pm$ 0.51	74.44 $\pm$ 0.64	82.17 $\pm$ 0.47	82.10 $\pm$ 0.45	76.91 $\pm$ 0.58
BERT-prompt	78.03 $\pm$ 0.55	77.83 $\pm$ 0.63	71.44 $\pm$ 0.78	81.07 $\pm$ 0.67	80.99 $\pm$ 0.62	75.51 $\pm$ 0.78
CPS-Rater	67.74 $\pm$ 2.04	67.18 $\pm$ 2.15	57.23 $\pm$ 2.50	57.65 $\pm$ 1.58	55.39 $\pm$ 1.77	42.76 $\pm$ 1.94
CPS-Rater (BERT)	71.58 $\pm$ 1.44	71.01 $\pm$ 1.49	62.66 $\pm$ 1.51	77.85 $\pm$ 1.08	77.56 $\pm$ 1.13	71.10 $\pm$ 1.27
BERT-seq	79.81 $\pm$ 0.71	79.82 $\pm$ 0.71	73.94 $\pm$ 0.93	82.70 $\pm$ 0.64	82.65 $\pm$ 0.58	77.63 $\pm$ 0.76
CAP4CPS	81.04$\pm$0.54	80.78$\pm$0.55	75.33$\pm$0.72	83.27$\pm$0.36	83.17$\pm$0.38	78.37$\pm$0.48

ten times and then computed the mean and standard deviation of the results across the ten runs, as presented in Table 6. Based on the analysis of these results, we derived several key findings:

1. The proposed CAP4CPS model outperforms all baselines on both datasets. Compared to the second-best model, CAP4CPS improves Accuracy by 1.00%, W-F1 by 0.81%, and Kappa by 1.20% on the Three-Resistor dataset (compared to BERT-finetune), and Accuracy by 0.69%, W-F1 by 0.63%, and Kappa by 0.95% on the Mondrian Geometric Fantasy dataset (compared to BERT-seq). These results indicate that CAP4CPS is more effective for CPS automated coding by modeling multi-dimensional context.
2. Methods based on pre-trained language models demonstrate substantially superior automated coding performance compared with other approaches. Most traditional machine learning methods that rely on N-gram representations for sentence features perform relatively poorly: the Kappa values of Naive Bayes, KNN, and CPS-Rater are all below 0.7, and only Random Forest achieves a Kappa of 0.7 on the Mondrian Geometric Fantasy dataset. Deep learning methods automatically learn sentence representations from data and thus exhibit stronger representational capacity than direct N-gram features. The CNN, GRU, and LSTM models all achieve Kappa values above 0.7 on Mondrian Geometric Fantasy; however, their Kappa values on Three-Resistor remain below 0.7. In contrast, methods based on pre-trained language models, which are trained on large-scale corpora, produce the most semantically rich sentence representations. Consequently, BERT-finetune, BERT-prompt, BERT-seq, and CAP4CPS all achieve Kappa values above 0.7 on both datasets.
3. The performance gains of CAP4CPS are not only attributable to the rich semantic representations obtained from BERT pretraining, but also to the design of the model architecture. To examine whether the improvement of CAP4CPS stems solely from BERT's semantic representations or also from the architectural design, we further included Random Forest, KNN, and CPS-Rater models that use BERT mean-pooled sentence embeddings as features. For KNN and CPS-Rater, the semantic representations provided by BERT substantially improve coding performance; in particular, CPS-Rater achieves a Kappa improvement of 66.3% on the Mondrian Geometric Fantasy dataset. However, we observe that Random Forest performance actually degrades when using BERT mean-pooled sentence embeddings. This may be because Random Forest is not well suited to dense features with high inter-dimensional correlations. When comparing these models using BERT mean-pooled sentence embeddings with methods that directly fine-tune BERT or apply prompt-based BERT, we find that

the former still perform markedly worse than the latter. These findings indicate that although BERT pretraining enables the learning of rich semantic representations, designing an appropriate model architecture to effectively exploit such representations is also crucial for improving automated coding performance.

4. Context-aware models achieve relatively strong performance. Among them, CPS-Rater using BERT mean-pooled sentence embeddings outperforms all other machine learning models on the Three-Resistor dataset. On Mondrian Geometric Fantasy, its performance is slightly worse than that of KNN, but still superior to the other machine learning models. BERT-seq ranks second on Mondrian Geometric Fantasy and third on Three-Resistor. One possible reason why BERT-seq does not consistently outperform BERT-finetune is that contextual information based on fixed infusion may sometimes introduce noise. In contrast, CAP4CPS models cognitive and social contextual information separately through prompt construction and incorporates an attention-based fusion mechanism, enabling a more flexible and effective understanding of contextual relationships.

Overall, this evidence indicates that CAP4CPS achieves superior performance on CPS automated coding tasks not only due to the rich semantic representations provided by pre-trained language models, but also owing to its well-designed model architecture. By constructing prompts to model multidimensional contextual information and fusing them through an attention-based mechanism, CAP4CPS is able to more effectively capture contextual relationships.

6.4.2. Ablation study (RQ2)

We conducted ablation studies to examine the contribution of each module in CAP4CPS. There are the variants of CAP4CPS that we consider, where “w/o” denotes CAP4CPS without a specific module:

- **w/o Context** means that the final prompt of CAP4CPS does not incorporate the multi-dimensional contextual information C_i^c and C_i^s .
- **w/o Attention** means that CAP4CPS does not consider the dependency between cognitive ability and social ability. We remove the multi-head attention mechanism and simply concatenate the representations of cognitive and social abilities.
- **w/o All** means without both context and the attention mechanism.

Table 7 presents a comparison between CAP4CPS and its ablated variants on the Three-Resistor and Mondrian Geometric Fantasy datasets. The results reveal the following findings:

Table 7

Ablation study of CAP4CPS. We evaluate variants of CAP4CPS in terms of Accuracy, W-F1 and Kappa ($\times 100$).

Model	Three-resistor			Mondrian Geometric Fantasy		
	Accuracy	Weighted-F1	Kappa	Accuracy	Weighted-F1	Kappa
CAP4CPS	81.04$\pm$0.54	80.78$\pm$0.55	75.33$\pm$0.72	83.27$\pm$0.36	83.17$\pm$0.38	78.37$\pm$0.48
w/o Context	80.73 $\pm$ 0.61	80.54 $\pm$ 0.54	74.96 $\pm$ 0.74	82.33 $\pm$ 0.67	82.13 $\pm$ 0.65	77.07 $\pm$ 0.85
w/o Attention	80.96 $\pm$ 0.54	80.69 $\pm$ 0.63	75.22 $\pm$ 0.80	83.23 $\pm$ 0.43	83.16 $\pm$ 0.43	78.36 $\pm$ 0.56
w/o All	80.63 $\pm$ 0.52	80.48 $\pm$ 0.53	74.90 $\pm$ 0.69	82.27 $\pm$ 0.59	82.09 $\pm$ 0.53	76.99 $\pm$ 0.74

- (1) The full CAP4CPS model achieves the highest performance, and removing any module leads to a decrease in performance. This demonstrates that each component of CAP4CPS is essential and contributes to improving the automated coding of CPS.
- (2) Removing the multi-dimensional contextual information results in the most significant performance decline, indicating that the proposed method effectively captures the relationship between contextual cues and the target message.
- (3) Eliminating the multi-head attention mechanism also leads to a performance drop. While simple concatenation captures some shallow dependencies between cognitive and social abilities, the multi-head attention mechanism enables each attention head to learn distinct aspects of the data, thereby modeling more complex and hierarchical relationships.

In conclusion, the strong performance of the CAP4CPS model is primarily attributed to its integration of multi-dimensional contextual information and its ability to model dependencies between cognitive and social abilities using the multi-head attention mechanism.

6.4.3. Sensitivity of parameters (RQ3)

In this section, we analyze the sensitivity of the CAP4CPS model to the parameters w^c and w^s , which define the window sizes for cognitive and social context, respectively. A key contributor to the strong performance of the proposed CAP4CPS model is its effective utilization of contextual information, with w^c and w^s serving as crucial parameters in constructing this context. Specifically, w^c governs the modeling of an individual's problem solving process, while w^s governs the modeling of interactions with others. For the experimental setup, both w^c and w^s are searched over the range $\{0, 1, 2, 3, 4, 5\}$, resulting in 36 combinations. Each combination is evaluated over 10 runs, and the results are averaged based on the Kappa score. The results are presented in Fig. 8.

Based on the results, we observe the following: (1) General trend: Larger values of both w^c and w^s generally lead to better performance. When these values are small, the model relies more heavily on the

current message for CPS skill recognition. In contrast, larger values enable the model to incorporate more contextual information, thereby enhancing its understanding of the dialogue environment in which the current message occurs. (2) Saturation effect: When both w^c and w^s are large, their specific values have a relatively minor impact on performance. In CPS dialogues, participants typically respond to recently sent messages; thus, beyond a certain window size, additional context does not yield further improvements.

These findings highlight the importance of selecting appropriate values for w^c and w^s to balance cognitive and social contexts, which is critical for CAP4CPS's performance in automated coding. Nevertheless, once these values exceed a certain threshold, the model's performance becomes relatively insensitive to further changes, demonstrating robustness to parameter selection.

6.4.4. Coding results analysis

To more thoroughly evaluate the automated coding performance of our proposed CAP4CPS model, we present its performance on the classification of individual CPS skills and analyze the associated misclassifications. Fig. 9 shows the normalized confusion matrices of the labels predicted by CAP4CPS on the two datasets. In each matrix, the entry at row i and column j represents the proportion of samples with true label i that were predicted as label j . The diagonal elements indicate the recall for each CPS skill.

On the Three-Resistor dataset, the recall rates for five skills—SSI, SESU, SMC, SN, and CEC—are relatively high, while those for CP, CMC, and CRF are comparatively low. We further analyze the three skills with lower predictive performance. For the CP skill, 22% of the samples are misclassified as CEC. CP involves discussions about plans for solving tasks, whereas CEC refers to discussions following task execution. Although both pertain to task-related actions, CP typically precedes the action, while CEC follows it. One example of a misclassified CP utterance is “take it at 330 again for a sec,” which expresses a plan for the next step. However, the model may have interpreted the word “again” as an indication that the action had already occurred. For CMC and CRF,

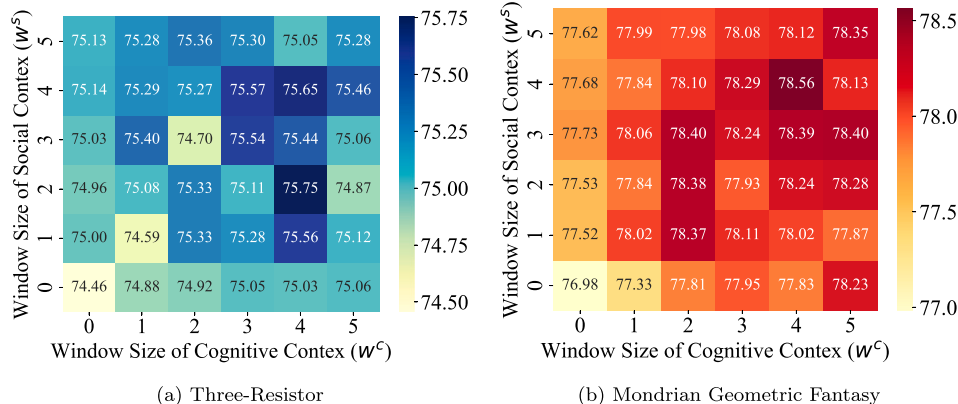


Fig. 8. Kappa of CAP4CPS under different window sizes for cognitive and social contexts ($\times 100$).

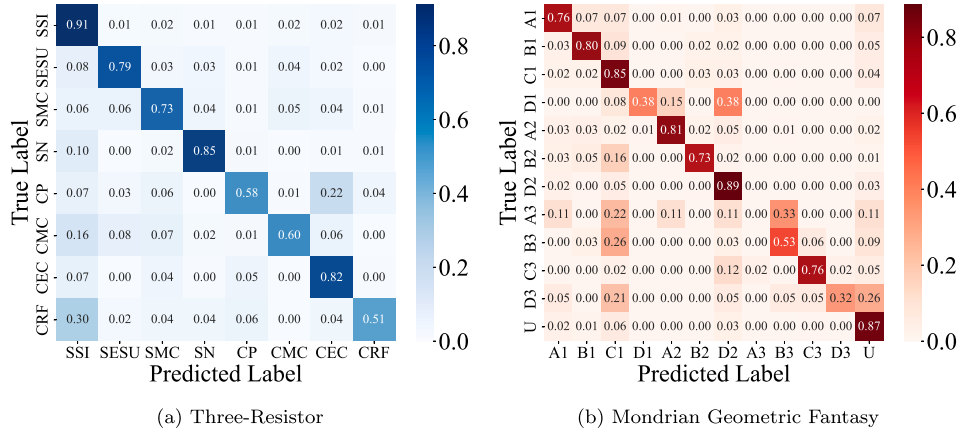


Fig. 9. Normalized confusion matrices of CAP4CPS on two datasets.

a significant proportion of samples are misclassified as SSI—16% for CMC and 30% for CRF. CMC refers to descriptions of task progress, and CRF denotes the formulation of task goals, while SSI involves sharing information. Although SSI belongs to a different skill dimension, some expressions in CMC and CRF resemble those in SSI. For instance, a misclassified CMC example is “we haven’t actually fixed the circuit yet,” and a misclassified CRF example is “and there’s a fourth resistor that we know nothing about.” While both sentences convey known information, the former focuses on task progress and the latter on summarizing shared knowledge. Another contributing factor to the misclassification is class imbalance: CMC and CRF have relatively few samples in the dataset, whereas SSI is far more prevalent.

On the Mondrian Geometric Fantasy dataset, the recall rates of the four skills D1, A3, B3, and D3 are relatively low. Notably, these four skills are exactly the four least frequent ones in the dataset, which indicates that CAP4CPS still has limitations in recognizing low-frequency skills. In addition, CAP4CPS tends to misclassify low-frequency skills as other skills that are semantically similar. Specifically, 38% of D1 instances are misclassified as D2. Both D1 and D2 are related to monitoring and reflection, but D1 focuses on team consensus, whereas D2 emphasizes the completion status of specific tasks. An example of such a misclassification is “that’s wrong, you’re not allowed to cross the white line.” The model may struggle with this kind of indirect expression, where the speaker corrects an understanding of overall rules by evaluating a specific behavior. The recognition performance of A3 is particularly poor: 33% of A3 instances are misclassified as B3, and another 22% as C1. A3 refers to explicitly specifying each person’s concrete responsibilities in problem solving; therefore, it typically involves both role assignment and concrete task execution. If the model attends to only one of these aspects, misclassification is likely to occur. For example, “I’ll take over at the end.” is misclassified as B3 because the model focuses only on role assignment and fails to capture that “at the end” refers to a specific stage of task execution. Similarly, “I am responsible for the lower-left corner” is misclassified as C1 because the model focuses on the specific task region while ignoring the aspect of responsibility assignment. For B3, 26% of instances are misclassified as C1. B3 is intended to describe high-level role allocation. A misclassified example is “you do it, I’ll tell you”. Although no concrete actions are specified, the model may focus on the words “do” and “tell” and thus interpret the utterance as referring to specific actions. For D3, 21% of instances are misclassified as C1. D3 is supposed to capture monitoring of responsibility fulfillment. Since responsibility fulfillment often involves concrete actions, it is easily mistaken for C1. An example is “make sure you understand it before you talk”, which should be interpreted as reminding someone to be accountable for their speech, whereas the model may focus only on the speaking action itself. Finally, 26% of D3 instances are misclassified as

U. These cases mainly involve expressions that are overly implicit. For example, “isn’t there another person?” appears irrelevant on the surface, but its underlying meaning is that one teammate has neither acted nor spoken for a long time and thus has failed to fulfill their responsibilities.

Overall, CAP4CPS faces several major challenges in automated coding. First, it often relies heavily on lexical features, which limits its ability to distinguish between skills with similar meanings that are expressed through subtle linguistic differences. Second, when the skill reflected in a sentence requires integrating information across the entire sentence, and different parts of the sentence may correspond to different skills, CAP4CPS tends to focus on local segments rather than the global context. Third, when expressions are highly implicit and their underlying meanings deviate from the literal meanings associated with the sentence labels, CAP4CPS may fail to capture the intended semantics. Finally, data imbalance also poses a significant challenge for CAP4CPS.

7. Discussion

With the growing importance of CPS skills across various domains, the assessment and development of these abilities have garnered increasing attention. Computer-simulated CPS tasks offer rich data for evaluating such skills, and a critical step in analyzing these data is encoding them into distinct CPS skill categories. Given the time-consuming and labor-intensive nature of traditional manual coding methods, numerous automated coding approaches have been proposed. However, many existing methods fail to account for the interdependencies between actions in the CPS process. To address this limitation, we propose a novel method—Context-Aware Prompting for CPS (CAP4CPS)—which effectively captures these dependencies to enhance the accuracy of automated coding. In this section, we summarize our key findings, identify limitations in current research, and outline directions for future work.

7.1. Main findings

The comparative experiments between CAP4CPS and other baseline models demonstrate CAP4CPS’s superior performance in CPS automated coding tasks. We evaluated CAP4CPS and the baseline models on two datasets: Three-Resistor and Mondrian Geometric Fantasy. CAP4CPS achieved the highest accuracy, weighted F1 score, and Cohen’s kappa across all models, indicating its effectiveness for CPS automated coding. Furthermore, models based on pre-trained language models consistently outperformed traditional machine learning and deep learning methods on both datasets. This suggests that pre-trained language models not only improve performance but also offer better adaptability across different tasks. Notably, the strong performance of both BERT-seq and CAP4CPS underscores the value of integrating pre-trained models

with contextual modeling. Finally, a comparison between Prompt-BERT and CAP4CPS—both of which utilize prompts—reveals that incorporating contextual information into prompts is a promising strategy for enhancing automated coding performance.

The ablation study reveals that incorporating multi-dimensional contextual information and modeling the dependencies between the cognitive and social dimensions both enhance automated coding performance. First, removing contextual information from both dimensions in CAP4CPS led to a significant decrease in performance, indicating the critical role of context in CPS processes. Without context, the model must rely solely on isolated individual behaviors, lacking awareness of the surrounding situation. Second, replacing the multi-head attention mechanism with simple concatenation caused a slight performance decline, suggesting that multi-head attention is more effective in capturing complex and hierarchical relationships between cognitive and social dimensions.

By analyzing the effect of context length in the cognitive and social dimensions on the CAP4CPS model's performance, we found that CAP4CPS is highly robust to variations in context length. First, we observed that, overall, longer contexts lead to better performance than very short ones, as extended context helps the model better understand the CPS process. Second, the impact of context length differs between the cognitive and social dimensions. A shorter context in the cognitive dimension has a more detrimental effect on CAP4CPS's performance. This is because, in conversations, speakers typically respond only to the most recent utterances from others, whereas their own utterances often form a coherent narrative across multiple consecutive statements. Finally, we found that as long as the context is not extremely short, the specific length has minimal influence on CAP4CPS's performance.

7.2. Limitations and future work

Like many other models, CAP4CPS has several limitations that warrant further investigation. First, this study employs BERT as the pre-trained language model and adopts a fixed-prompt PLM tuning approach. In recent years, large language models (LLMs) have demonstrated impressive performance and strong zero-shot capabilities. We aim to explore the potential of using LLMs for automated coding in a zero-shot setting. Second, the proposed model is trained on a specific task dataset, achieving satisfactory performance only on that task, which limits its generalizability. In future work, we plan to develop models capable of performing automated coding across multiple CPS tasks. Third, our current model uses a fixed-window approach for context modeling. Although it exhibits strong robustness to variations in window size, optimal performance still depends on manually selecting an appropriate window length. Therefore, we intend to explore adaptive context selection methods that allow the model to automatically determine the optimal context length. Lastly, CAP4CPS exhibits relatively low recognition accuracy for low-frequency samples when dealing with extremely imbalanced data distributions. In future work, we plan to design dedicated algorithms to improve the recognition accuracy of low-frequency samples while maintaining overall performance as much as possible.

8. Conclusion

In this study, we proposed Context-Aware Prompting for CPS (CAP4CPS), a novel framework designed to improve the automated coding of CPS skills by explicitly modeling contextual dependencies within the CPS process. By extracting dialogue histories along both cognitive and social dimensions and incorporating them into prompts for a pre-trained language model, CAP4CPS facilitates a more accurate understanding of the situational context behind each behavior. The model further integrates these dimensions into a unified representation of CPS competency for skill prediction. Extensive experiments on two CPS datasets validate the effectiveness and robustness of CAP4CPS.

CRedit authorship contribution statement

Mengxiao Zhu: Writing – review & editing, Resources, Project administration, Conceptualization. **Li Feng:** Writing – original draft, Visualization, Methodology, Formal analysis. **Xin Wang:** Writing – review & editing. **Wei Huang:** Writing – review & editing, Supervision.

Funding

This work was supported by the National Education Examinations Authority [grant number GJK2024005].

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This research was also supported by the advanced computing resources provided by the Supercomputing Center of the USTC.

Data availability

The data that support the findings of this study are available from the corresponding author upon reasonable request.

References

- Andrews-Todd, J., & Forsyth, C. M. (2020). Exploring social and cognitive dimensions of collaborative problem solving in an open online simulation-based task. *Computers in Human Behavior*, 104, 105759. <https://doi.org/10.1016/j.chb.2018.10.025>
- Andrews-Todd, J., Jackson, G. T., & Kurzum, C. (2019). Collaborative problem solving assessment in an online Mathematics task. *ETS Research Report Series*, 2019, 1–7. <https://doi.org/10.1002/ets2.12260>
- Andrews-Todd, J., & Kerr, D. (2019). Application of ontologies for assessing collaborative problem solving skills. *International Journal of Testing*, 19, 172–187. <https://doi.org/10.1080/15305058.2019.1573823>
- Aslan, S., Alyuz, N., Li, B., Durham, L. M., Shi, M., Sharma, S., & Nachman, L. (2025). An early investigation of collaborative problem solving in conversational AI-mediated learning environments. *Computers and Education: Artificial Intelligence*, 8, 100393. <https://doi.org/10.1016/j.caeai.2025.100393>
- Ben-David, E., Oved, N., & Reichart, R. (2022). PADA: Example-based prompt learning for on-the-fly adaptation to unseen domains. *Transactions of the Association for Computational Linguistics*, 10, 414–433. https://doi.org/10.1162/tacl_a.00468
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., ... Amodei, D. (2020). Language models are few-shot learners. In *Advances in neural information processing systems* (pp. 1877–1901). Curran Associates, Inc.
- Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). Empirical evaluation of gated recurrent neural networks on sequence modeling. arXiv preprint [arXiv:1412.3555](https://arxiv.org/abs/1412.3555). <https://doi.org/10.48550/arXiv.1412.3555>
- Cui, G., Hu, S., Ding, N., Huang, L., & Liu, Z. (2022). Prototypical verbalizer for prompt-based few-shot tuning. In S. Muresan, P. Nakov, & A. Villavicencio (Eds.), *Proceedings of the 60th annual meeting of the association for computational linguistics (volume 1: Long papers)* (pp. 7014–7024). Dublin, Ireland: Association for Computational Linguistics. <https://doi.org/10.18653/v1/2022.acl-long.483>
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In J. Burstein, C. Doran, & T. Solorio (Eds.), *Proceedings of the 2019 conference of the north American chapter of the association for computational linguistics: Human language technologies, volume 1 (long and short papers)* (pp. 4171–4186). Minneapolis, Minnesota: Association for Computational Linguistics. <https://doi.org/10.18653/v1/N19-1423>
- Flor, M., & Andrews-Todd, J. (2022). Towards automatic annotation of collaborative problem-solving skills in technology-enhanced environments. *Journal of Computer Assisted Learning*, 38, 1434–1447. <https://doi.org/10.1111/jcal.12689>
- Flor, M., Yoon, S.-Y., Hao, J., Liu, L., & von Davier, A. (2016). Automated classification of collaborative problem solving interactions in simulated science tasks. In J. Tetreault, J. Burstein, C. Leacock, & H. Yannakoudakis (Eds.), *Proceedings of the 11th workshop on innovative use of NLP for building educational applications* (pp. 31–41). San Diego, CA: Association for Computational Linguistics. <https://doi.org/10.18653/v1/W16-0504>
- Graesser, A. C., Fiore, S. M., Greiff, S., Andrews-Todd, J., Foltz, P. W., & Hesse, F. W. (2018). Advancing the science of collaborative problem solving. *Psychological Science in the Public Interest*, 19, 59–92. <https://doi.org/10.1177/1529100618808244>

- Han, X., Zhang, Z., Ding, N., Gu, Y., Liu, X., Huo, Y., Qiu, J., Yao, Y., Zhang, A., Zhang, L., Han, W., Huang, M., Jin, Q., Lan, Y., Liu, Y., Liu, Z., Lu, Z., Qiu, X., Song, R. ... Zhu, J. (2021). Pre-trained models: Past, present and future. *AI Open*, 2, 225–250. <https://doi.org/10.1016/j.aiopen.2021.08.002>
- Hao, J., Chen, L., Flor, M., Liu, L., & Von Davier, A. A. (2017). CPS-rater: Automated sequential annotation for conversations in collaborative problem-solving activities. *ETS Research Report Series*, 2017, 1–9. <https://doi.org/10.1002/ets2.12184>
- Hao, J., Liu, L., Kyllonen, P., Flor, M., & Von Davier, A. A. (2019). Psychometric considerations and a general scoring strategy for assessments of collaborative problem solving. *ETS Research Report Series*, 2019, 1–17. <https://doi.org/10.1002/ets2.12276>
- Hesse, F., Care, E., Buder, J., Sassenberg, K., & Griffin, P. (2015). A framework for teachable collaborative problem solving skills. In P. Griffin, & E. Care (Eds.), *Assessment and teaching of 21st century skills* (pp. 37–56). Dordrecht: Springer Netherlands. https://doi.org/10.1007/978-94-017-9395-7_2
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9, 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- Horwitz, P., Von Davier, A., Chamberlain, J., Koon, A., Andrews, J., & McIntyre, C. (2017). Teaching teamwork: Electronics instruction in a collaborative environment. *Community College Journal of Research and Practice*, 41, 341–343. <https://doi.org/10.1080/10668926.2016.1275886>
- Kim, Y. (2014). Convolutional neural networks for sentence classification. In A. Moschitti, B. Pang, & W. Daelemans (Eds.), *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)* (pp. 1746–1751). Doha, Qatar: Association for Computational Linguistics. <https://doi.org/10.3115/v1/D14-1181>
- Lester, B., Al-Rfou, R., & Constant, N. (2021). The power of scale for parameter-efficient prompt tuning. In M. F. Moens, X. Huang, L. Specia, & S. W. T. Yih (Eds.), *Proceedings of the 2021 conference on empirical methods in natural language processing* (pp. 3045–3059). Online and Punta Cana, Dominican Republic: Association for Computational Linguistics. <https://doi.org/10.18653/v1/2021.emnlp-main.243>
- Li, X. L., & Liang, P. (2021). Prefix-tuning: Optimizing continuous prompts for generation. In C. Zong, F. Xia, W. Li, & R. Navigli (Eds.), *Proceedings of the 59th annual meeting of the association for computational linguistics and the 11th international joint conference on natural language processing (volume 1: Long papers)* (pp. 4582–4597). Online: Association for Computational Linguistics. <https://doi.org/10.18653/v1/2021.acl-long.353>
- Liu, L., Hao, J., von Davier, A. A., Kyllonen, P. C., & Zapata-Rivera, D. (2016). A tough nut to crack: Measuring collaborative problem solving PISA CPS GBA. In Y. Rosen, S. Ferrara, M. Mosharraf, & J. Keengwe (Eds.), *Handbook of research on technology tools for Real-World skill development Advances in higher education and professional development*. (pp. 344–359). IGI Global. <https://doi.org/10.4018/978-1-4666-9441-5>
- Liu, P., Yuan, W., Fu, J., Jiang, Z., Hayashi, H., & Neubig, G. (2023). Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, 55, 1–35. <https://doi.org/10.1145/3560815>
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V., 2019. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*. <https://doi.org/10.48550/arXiv.1907.11692>
- Mislevy, R. J., Steinberg, L. S., & Almond, R. G. (2003). Focus article: On the structure of educational assessments. *Measurement: Interdisciplinary Research & Perspective*, 1, 3–62. https://doi.org/10.1207/S15366359MEA0101_02
- Och, F. J., Gildea, D., Khudanpur, S., Sarkar, A., Yamada, K., Fraser, A., Kumar, S., Shen, L., Smith, D., Eng, K., Jain, V., Jin, Z., & Radev, D. (2004). A smorgasbord of features for statistical machine translation. In *Proceedings of the human language technology conference of the north American chapter of the association for computational linguistics: HLT-NAACL 2004* (pp. 161–168). Boston, Massachusetts, USA: Association for Computational Linguistics.
- OECD. (2017). *PISA 2015 assessment and analytical framework: Science, reading, mathematics, financial literacy and collaborative problem solving*. PISA, OECD. <https://doi.org/10.1787/9789264281820-en>
- Pugh, S. L., Rao, A., Stewart, A. E. B., & D'Mello, S. K. (2022). Do speech-based collaboration analytics generalize across task contexts? In *Lak22: 12th international learning analytics and knowledge conference* (pp. 208–218). New York, NY, USA: Association for Computing Machinery. <https://doi.org/10.1145/3506860.3506894>
- Pugh, S. L., Subburaj, S. K., Rao, A. R., Stewart, A. E. B., Andrews-Todd, J., & D'Mello, S. K. (2021). Say what? Automatic modeling of collaborative problem solving skills from student speech in the wild. In *Proceedings of the 14th international conference on educational data mining*. International Educational Data Mining Society.
- Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I., 2018. Improving language understanding by generative pre-training. OpenAI Technical Report. https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf.
- Schick, T., & Schütze, H. (2021a). Exploiting cloze-questions for few-shot text classification and natural language inference. In P. Merlo, J. Tiedemann, & R. Tsarfaty (Eds.), *Proceedings of the 16th conference of the european chapter of the association for computational linguistics: Main volume* (pp. 255–269). Online: Association for Computational Linguistics. <https://doi.org/10.18653/v1/2021.eacl-main.20>
- Schick, T., Schütze, H. (2021b). It's not just size that matters: Small language models are also few-shot learners. In Toutanova, K., Rumshisky, A., Zettlemoyer, L., Hakkani-Tur, D., Beltagy, I., Bethard, S., Cotterell, R., Chakraborty, T., Zhou, Y. (Eds.), *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Association for Computational Linguistics, Online. pp. 2339–2352. <https://doi.org/10.18653/v1/2021.naacl-main.185>.
- Song, X., Huang, L., Xue, H., & Hu, S. (2022). Supervised prototypical contrastive learning for emotion recognition in conversation. In *Proceedings of the 2022 conference on empirical methods in natural language processing* (pp. 5197–5206). Abu Dhabi, United Arab Emirates: Association for Computational Linguistics. <https://doi.org/10.18653/v1/2022.emnlp-main.347>
- Stewart, A. E. B., Vrzakova, H., Sun, C., Yonehiro, J., Stone, C. A., Duran, N. D., Shute, V., & D'Mello, S. K. (2019). I say, you say, we say: Using spoken language to model socio-cognitive processes during Computer-Supported collaborative problem solving. *Proceedings of the ACM on Human-Computer Interaction*, 3, 1–19. <https://doi.org/10.1145/3359296>
- Stewart, A. E. B., Keirn, Z., & D'Mello, S. K. (2021). Multimodal modeling of collaborative problem-solving facets in triads. *User Modeling and User-Adapted Interaction*, 31, 713–751. <https://doi.org/10.1007/s11257-021-09290-y>
- Sun, C., Shute, V. J., Stewart, A., Yonehiro, J., Duran, N., & D'Mello, S. (2020). Towards a generalized competency model of collaborative problem solving. *Computers & Education*, 143, 103672. <https://doi.org/10.1016/j.compedu.2019.103672>
- Tan, S. C., Lee, A. V. Y., & Lee, M. (2022). A systematic review of artificial intelligence techniques for collaborative learning over the past two decades. *Computers and Education: Artificial Intelligence*, 3, 100097. <https://doi.org/10.1016/j.caeai.2022.100097>
- Xu, Q., Ge, Z., Zhu, M., Wang, J., & Zhang, M. (2024). Designing an online jigsaw puzzle task to assess collaborative problem-solving skills. In *2024 IEEE integrated STEM education conference (ISEC)* (pp. 1–8). Princeton, NJ, USA: IEEE. <https://doi.org/10.1109/ISEC61299.2024.10664914>
- Zhou, C., He, J., Ma, X., Berg-Kirkpatrick, T., & Neubig, G. (2022). Prompt consistency for zero-shot task generalization. In Y. Goldberg, Z. Kozareva, & Y. Zhang (Eds.), *Findings of the association for computational linguistics: EMNLP 2022* (pp. 2613–2626). Abu Dhabi, United Arab Emirates: Association for Computational Linguistics. <https://doi.org/10.18653/v1/2022.findings-emnlp.192>
- Zhu, M., Wang, X., Wang, X., Chen, Z., & Huang, W. (2024). Application of prompt learning models in identifying the collaborative problem solving skills in an online task. *Proceedings of the ACM on Human-Computer Interaction*, 8, 1–23. <https://doi.org/10.1145/3686981>